

プログラミング入門2

第3回

式文

代入式

論理演算子

break文、continue文

篠埜 功

今日の内容

- 式文
- 代入式
- 論理演算子
- break文、continue文

式文(教科書p. 29, p. 101)

例えば、文

$x=1$;

は、 $x=1$ が代入式であり、その次にセミコロンが書かれているという構造をしている。式の次にセミコロン;を書くと文になる。これを式文と言う。

式文の構文1

式;

式文 e ; の意味

式 e を評価する。評価結果は捨てる。

式文は文であり、文が書けるところならどこにでも書くことができる。

式文は、式の評価中に変数への代入や画面への出力などが起こる場合に用いる。 $1+2$;なども式文であり文であるが、このような文は書く意味がない。

式文の構文2：空文（教科書p. 101）

何もしない文として、空文というものがある。空文も式文である。

式文の構文2（空文）：

；

空文；の意味

何もしない

こんな構文を使う機会はないように思うかもしれないが、使うこともある。

これまでにでてきた式文の例

- 代入式のあとにセミコロン

`x = 1+2 ;`

- 関数呼び出し式のあとにセミコロン(関数の回でもう一度説明する)

`printf ("test") ;`

`scanf ("%d", &x) ;`

- 赤字の`x=1+2`, `printf("test")`, `scanf("%d",&x)`の部分はそれ自体式であり、値を持つ。(式文ではそれは捨てられるが。)

代入式(教科書 p. 29)

代入式の構文

変数名 = 式

(代入式の構文は後で(配列やポインタの
回で)拡張する)

代入式 $x = e$ の意味

まず式 e を評価し、その結果の値を変
数 x に代入する。代入式の値は変数 x
に代入された値である。

(注意) 代入式 $x=e$ の値は、式 e の値とは必ずしも同じではない。例えば、

```
int x;
```

```
x = 2.95;
```

のような例では、 x には2が代入されるので、代入式 $x=2.95$ の値は2である。

代入式がネストされたプログラム例1

```
#include <stdio.h>
int main (void) {
    int x;
    int y;
    x = y = 2;
    printf ("x=%d, y=%d\n", x, y);
    return 0;
}
```

赤字の $y=2$ の部分は代入式であり、
外側の代入式 $x=y=2$ の右辺を成
している。(代入式のネスト)

代入演算子 $=$ は右結合である。(教科書 p. 220)
つまり、式 $x=y=2$ は、式 $x=(y=2)$ と等価である。

代入式がネストされたプログラム例2 (打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    double x;
    int y;
    x = y = 1.5;
    printf ("x=%f, y=%d¥n", x, y);
    return 0;
}
```

代入式`y=1.5`においては、`y`が`int`型なので`1.5`が`int`型に変換され、`1`が`y`に代入される。
よって、代入式`y=1.5`の値は`1`である。次に`x=1`が評価され、`x`が`double`型なので、`1`が`double`型に変換され、`1.0`が`x`に代入される。

よくある間違い(教科書 p. 65)

```
#include <stdio.h>
int main (void) {
    int x;
    x = 1;
    if (x=2) {
        printf ("xの値は2です¥n");
    } else {
        printf ("xの値は2ではありません¥n");
    }
    return 0;
}
```

赤字のx=2の部分は代入式であり、xに2が代入される。よって代入式x=2の値は2である。if文の条件式の値が2であり、0ではないので、最初のprintfが実行されることになる。

x==2と書くつもりが、x=2と書いてしまうということがよくあるので、注意する。

論理AND演算子 &&

/* キーボードから月を読み込み、その月が春かどうかを判定
ただし3月から5月を春とする。 */

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int month;
```

```
    printf ("月を入力してください: ");
```

```
    scanf ("%d", &month);
```

```
    printf ("%d月は", month);
```

```
    if (month >= 3) {
```

```
        if (month <= 5)
```

```
            printf ("春です。¥n");
```

```
            else printf ("春ではありません。¥n");
```

```
    } else printf ("春ではありません。¥n");
```

```
    return 0;
```

```
}
```

3以上5以下の判定はif文の
入れ子で書けるが、if文一つ
で書きたい。

論理AND演算子 && (教科書 p. 62)

/* キーボードから月を読み込み、その月が春かどうかを判定。ただし3月から5月を春とする。 */

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int month;
```

```
    printf ("月を入力してください:");
```

```
    scanf ("%d", &month);
```

```
    printf ("%d月は", month);
```

```
    if (month >= 3 && month <= 5)
```

```
        printf ("春です。¥n");
```

```
    else printf ("春ではありません。¥n");
```

```
    return 0;
```

```
}
```

論理AND式(教科書 p. 62)

論理AND式の構文

式 && 式

論理AND式 $a \ \&\& \ b$ の意味

まず式aを評価し、その評価結果が0の場合は式bは評価せずに $a \ \&\& \ b$ の評価結果は0となる。式aの評価結果が0でない場合、式bを評価し、その値が0のときは0、そうでない場合は1が式 $a \ \&\& \ b$ の評価結果となる。

真偽が定まった時点で評価を終える。短絡評価(short circuit evaluation)という。(教科書 p. 64)

a, bに代入式やprintfなどを含む式を使うとプログラムが分かりにくくなる(ので避けた方がよい)。

論理AND式の評価に関する確認 (打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int x;
    x = 5;
    if (0 && (x=1))
        printf ("true¥n");
    else
        printf ("false¥n");
    printf ("x=%d¥n", x);
    return 0;
}
```

代入式 $x = 1$ は評価されない。

論理OR式 (教科書 p. 63)

論理OR式の構文

式 || 式

論理OR式 $a \ || \ b$ の意味

まず式aを評価し、その評価結果が0でない場合は式bは評価せずに $a \ || \ b$ の評価結果は1となる。式aの評価結果が0の場合、式bを評価し、その値が0のときは0、そうでない場合は1が式 $a \ || \ b$ の評価結果となる。

短絡評価 (short circuit evaluation) という。

a, bに代入式やprintfなどを含む式を使うのは避けた方がよい。

論理OR演算子||を使った例

/* キーボードから月を読み込み、その月が春かどうかを判定。ただし3月から5月を春とする。 */

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int month;
```

```
    printf ("月を入力してください: ");
```

```
    scanf ("%d", &month);
```

```
    printf ("%d月は", month);
```

```
    if (month == 3 || month == 4 || month == 5)
```

```
        printf ("春です。¥n");
```

```
    else printf ("春ではありません。¥n");
```

```
    return 0;
```

```
}
```

論理演算子の優先順位 (教科書 p. 221)

四則演算の演算子では、 $+$, $-$ より $*$, $/$ の優先順位が高い。

論理演算子の $\&\&$ は $||$ より優先順位が高い。

&&と||の優先順位を示す例

```
#include <stdio.h>
int main (void) {
    if (1 || 0 && 0)
        printf ("true¥n");
    else
        printf ("false¥n");
    return 0;
}
```

0 && 0が||の右側の引数と解釈されるので、条件部分が1となり、trueが表示される。

論理否定演算子! (教科書 p. 77)

否定式の構文

!式

否定式 !e の意味

式eを評価し、その評価結果が0のとき1、そうでない場合0が!eの評価結果である。

否定演算子を使った例

```
#include <stdio.h>
int main (void) {
    if (!0)
        printf ("true¥n");
    else
        printf ("false¥n");
    return 0;
}
```

!0の評価結果は1であり、trueが表示される。

ループの脱出(goto文による) (打ち込んで確認)

```
/* 5匹目を数えるところで終了 */  
#include <stdio.h>  
int main (void) {  
    int x;  
    x = 1;  
    while (x <= 10) {  
        if (x == 5) goto aaa;  
        printf ("羊が%d匹¥n", x);  
        x=x+1;  
    }  
    aaa:  
    return 0;  
}
```

ループの脱出 (break文による)

(教科書 p. 93, p. 103) (打ち込んで確認)

```
/* 5匹目を数えるところで終了 */
```

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int x;
```

```
    x = 1;
```

```
    while (x <= 10) {
```

```
        if (x == 5) break;
```

```
        printf ("羊が%d匹¥n", x);
```

```
        x=x+1;
```

```
    }
```

```
    return 0;
```

```
}
```

break文が実行されると、それが属する最も内側のループ (whileループ, forループ, do whileループ) の直後の部分へジャンプする。

break文はswitch文 (この講義では扱わない) を脱出するためにも使われる。

ループの残りの処理のスキップ (goto文による)(打ち込んで確認)

```
/* 5匹目だけ表示しない */  
#include <stdio.h>  
int main (void) {  
    int x;  
    x = 1;  
    while (x <= 10) {  
        if (x == 5) {  
            x=x+1;  
            goto aaa;  
        }  
        printf ("羊が%d匹¥n", x);  
        x=x+1;  
aaa:  
        ;  
    }  
    return 0;  
}
```

ここのセミicolonは空文。

ループの残りの処理のスキップ (continue文による)

(教科書 p. 93, p. 103) (打ち込んで確認)

```
/* 5匹目だけ表示しない */
```

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int x;
```

```
    x = 1;
```

```
    while (x <= 10) {
```

```
        if (x == 5) {
```

```
            x=x+1;
```

```
            continue;
```

```
        }
```

```
        printf ("羊が%d匹¥n", x);
```

```
        x=x+1;
```

```
    }
```

```
    return 0;
```

```
}
```

continue文が実行されると、それが属する最も内側のループ(whileループ, forループ, do whileループ)の最後の部分へジャンプする。

基本課題1

以下の例のように、キーボードから月を読み込み、その月が秋かどうかを判定して表示するプログラムを論理演算子を用いて書け。ただし9月から11月を秋とする。

[実行例]

```
$ ./a.out
```

```
月を入力してください: 3
```

```
3月は秋ではありません。
```

```
$ ./a.out
```

```
月を入力してください: 9
```

```
9月は秋です。
```

```
$
```


基本課題2

以下の例のように、2つの正の整数(正のint型の数)をキーボードから読み取り、2つの数の最小値(小さい方)がもう片方の数の約数かどうかを表示するということを繰り返すプログラムをbreak文を用いて作成せよ。ただし、毎回次のようなメッセージを出し、確認するようにせよ。[続ける:1, 終了:0]

[実行例]

\$./a.out

数値を入力: 5

数値を入力: 10

5は10の約数です。

[続ける:1, 終了:0] 1

数値を入力: 6

数値を入力: 3

3は6の約数です。

[続ける:1, 終了:0] 1

(実行例続き)

数値を入力: 7

数値を入力: 10

7は10の約数ではありません。

[続ける:1, 終了:0] 1

数値を入力: 5

数値を入力: 5

5は5の約数です。

[続ける:1, 終了:0] 0

\$

発展課題1

西暦の年数(1582年以降)をキーボードから読み取り、閏年かどうかを以下のように表示するプログラムを、論理演算子を用いて(if文1つだけ用いて)記述せよ。(1582年以降かどうかのチェックをする場合はif文2個用いてOKです。チェックしなくてもいいです。)閏年の定義は第1回発展課題2を参照せよ。

[実行例]

```
$ ./a.out
```

西暦(1582年以降)を入力してください: 2010

2010年は閏年ではありません。

```
$ ./a.out
```

西暦(1582年以降)を入力してください: 2000

2000年は閏年です。

```
$ ./a.out
```

西暦(1582年以降)を入力してください: 1900

1900年は閏年ではありません。

発展課題2

正の整数(正のint型の数)をキーボードから受け取り、各桁の数および桁数を以下のように表示するプログラムを書け。

[実行例]

\$./a.out

正の整数を入力してください: 234

1の位は4です。

10の位は3です。

100の位は2です。

234は3桁の数です。

\$

発展課題3

正の整数(正のint型の数)を2つキーボードから読みとり、それらの割り算を以下の例のように小数点以下50桁まで正確に行うプログラムを書け。(51桁目の四捨五入は行わない。)負の数が入力された場合の処理は自由にしてよい。ただし小数点以下50桁未満で割り切れたらそこで表示を終わらせるようにせよ。

[実行例1]

\$./a.out

正の整数(割られる数)を入力して下さい: 3

正の整数(割る数)を入力して下さい: 61

0.04918032786885245901639344262295081967213114754098

[実行例2]

\$./a.out

正の整数(割られる数)を入力して下さい: 500

正の整数(割る数)を入力して下さい: 16

31.25

\$

参考課題1

整数を2つキーボードから入力し、それらの和を表示するということを繰り返すプログラムを作成せよ。ただし、毎回以下のようなメッセージを出し、確認するようにせよ。

続ける場合は1を、終了する場合は0を入力してください:

[実行例]

\$./a.out

整数を一つ入力してください: 2

整数をもう一つ入力してください: 3

$2 + 3 = 5$

続ける場合は1を、終了する場合は0を入力してください: 1

整数を一つ入力してください: 34

整数をもう一つ入力してください: 45

$34 + 45 = 79$

続ける場合は1を、終了する場合は0を入力してください: 0

\$

参考課題1 解答例

```
#include <stdio.h>
int main (void) {
    int a;
    int b;
    int c;
    while (1) {
        printf ("整数を一つ入力してください: ");
        scanf ("%d", &a);
        printf ("整数をもう一つ入力してください: ");
        scanf ("%d", &b);
        printf ("%d + %d = %d¥n", a, b, a+b);
        printf ("続ける場合は1を、終了する場合は0を入力してください: ");
        scanf ("%d", &c);
        if (c==0) break;
    }
    return 0;
}
```

参考課題2

キーボードから得点を入力し、負の数が入力された時に、それまでに入力された得点(負の数を含まない)の平均点を表示するプログラムを作成せよ。

[実行例]

```
$ ./a.out
```

```
得点を入力して下さい: 30
```

```
得点を入力して下さい: 60
```

```
得点を入力して下さい: 65
```

```
得点を入力して下さい: -1
```

```
平均点は51.666667点です。
```

```
$
```

参考課題2 解答例

```
#include <stdio.h>
int main (void) {
    int x;
    int sum;
    int n;
    sum = 0;
    n = 0;
    while (1) {
        printf ("得点を入力して下さい: ");
        scanf ("%d", &x);
        if (x<0) break;
        sum = sum + x;
        n = n + 1;
    }
    printf ("平均点は%f点です。¥n", (double)sum/n);
    return 0;
}
```